



Motif Discovery in Harbinger



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Outline

A practical guide to discovering recurring patterns (motifs) and rare subsequences (discords) using the `harbinger` package in R. Examples cover Matrix Profile–based methods and symbolic representations (SAX/XSAX).

Matrix Profile (01–05)

- STAMP – approximate
- STOMP – exact
- SCRIMP – incremental
- PMP – multi-window
- VALMOD – variable length

Symbolic (06–07)

- SAX – standard alphabet
- XSAX – extended alphabet

Discords (08–09)

- Matrix Profile via STOMP (`hdis_mp`)
- Symbolic via SAX (`hdis_sax`)

Standard Workflow – harbinger Pipeline

All notebooks share the same pipeline. Only the model constructor changes.

Pipeline Steps

1. Load packages: `library(daltoolbox)` and `library(harbinger)`
2. Load data: `data(examples_motifs) → dataset <- examples_motifs$simple`
3. Visualize: `har_plot(harbinger(), dataset$serie)`
4. Define model: constructor varies per method (see each notebook)
5. Fit: `model <- fit(model, dataset$serie)`
6. Detect: `detection <- detect(model, dataset$serie)`
7. Evaluate: `evaluate(model, detection$event, dataset$event)`

```
# 1. Load packages
library(daltoolbox)
library(harbinger)

# 2. Load data
data(examples_motifs)
dataset <- examples_motifs$simple

# 3. Visualize
har_plot(harbinger(), dataset$serie)

# 4. Define model ← changes per notebook
model <- <constructor>(...)
```

```
# 5. Fit
model <- fit(model, dataset$serie)

# 6. Detect
detection <- detect(model,
dataset$serie)

# 7. Evaluate
evaluation <- evaluate(model,
detection$event, dataset$event)
print(evaluation$confMatrix)
```

01 – Motif Discovery with STAMP

STAMP – Scalable Time Series Anytime Matrix Profile

- Approximates the Matrix Profile via random sampling of subsequence pairs
- Can be interrupted at any time and still returns a valid result
- Complexity: $O(n^2 \log n)$; scalable for long time series
- Use when exact MP is too costly and approximation is sufficient
- Result: 2 TP, 8 FP, 1 FN

📄 Yeh et al. (2016). Matrix Profile I. IEEE ICDM.

```
# Constructor  
# w: window size  
# n_motifs: number of motifs  
model <- hmo_mp("stamp", 4, 3)
```

02 – Motif Discovery with STOMP

STOMP – Exact Matrix Profile Computation

- Guarantees the complete Matrix Profile without approximation
- Exploits sliding dot-product optimization; $O(n^2)$ time, $O(n)$ space
- STOMP is exact; STAMP is approximate but anytime — similar practical runtimes
- Use when precision is paramount and dataset fits in memory
- Result: 3 TP, 8 FP, 0 FN — perfect recall; 0.01 sec

 **Citation:** Zhu et al. (2016). Matrix Profile II. IEEE ICDM.

```
# Constructor
# w: window size
# n_motifs: number of motifs
model <- hmo_mp("stomp", 4, 3)
```

03 – Motif Discovery with SCRIMP

SCRIMP – Incremental & Anytime Matrix Profile

- Computes Matrix Profile using random diagonal ordering
- Interruptible — returns valid approximation at any stage
- Cache-friendly diagonal traversal; $O(n^2)$; converges to exact result
- Both SCRIMP and STAMP are anytime, but SCRIMP is more cache-friendly
- Result: 2 TP, 8 FP, 1 FN; 0.03 sec

📄 **Citation:** Zhu et al. (2018). Matrix Profile XI: SCRIMP++. IEEE ICDM.

```
# Constructor
# w: window size
# n_motifs: number of motifs
model <- hmo_mp("scrimp", 4, 3)
```

04 – Motif Discovery with Pan Matrix Profile

PMP – Pan Matrix Profile

- Simultaneously computes Matrix Profiles for a range of window sizes
- Eliminates the need to manually tune the window parameter
- Reveals motifs robust across different temporal scales
- Best for exploratory analysis where optimal scale is unknown
- Result: 3 TP, 4 FP, 0 FN — best precision among MP methods

📄 **Citation:** Linardi et al. (2020). VALMOD. ACM SIGMOD.

```
# Constructor  
# w: starting window size  
# n_motifs: number of motifs  
model <- hmo_mp("pmp", 4, 3)
```

05 – Motif Discovery with VALMOD

VALMOD – Variable-Length Motif Discovery

- Searches for motifs across a range [lmin, lmax] without a fixed window
- Guarantees exact motif discovery within the specified length range
- Supports streaming and incremental updates
- Uses length-normalized distance for robust comparisons
- VALMOD finds the single best motif; PMP consolidates all window sizes
- Result: 3 TP, 8 FP, 0 FN; 0.02 sec

📄 **Citation:** Linardi et al. (2018). Matrix Profile X: VALMOD. ACM SIGMOD.

```
# Constructor
# lmin: minimum window length
# n_motifs: number of motifs
model <- hmo_mp("valmod", 4, 3)
```

06 – Motif Discovery with SAX

SAX – Symbolic Aggregate approXimation

Converts numerical time series into symbolic words via 3 steps:

1. **Z-Normalization** — normalize to zero mean and unit std
2. **PAA** — divide into w segments; compute mean of each
3. **Symbolic Mapping** — map each PAA value to a letter using Gaussian breakpoints → word of length w
 - Motifs = most frequent symbolic words
 - Parameters: `hmo_sax(a, w, min_occ)` — a =alphabet, w =word length, min_occ =min occurrences
 - Result: 3 TP, 0 FP, 0 FN — perfect; motif "QST" at idx 25, 50, 75

```
# Constructor
# arg1: alphabet size (26 letters)
# arg2: word length (3)
# arg3: min occurrences (3)
model <- hmo_sax(26, 3, 3)
```

📖 Lin et al. (2007). *Data Mining and Knowledge Discovery*.

07 – Motif Discovery with XSAX

XSAX – Extended Symbolic Representation

Extends SAX by replacing the letter-only alphabet with an alphanumeric one (digits 0–9 + letters A–Z), increasing symbolic resolution for series with subtle amplitude variations.

- **SAX:** letters only (a=26), coarser resolution, detected word "QST"
- **XSAX:** digits + letters (a=37), finer resolution, detected word "0N0P0R" (2 chars per PAA segment)
- Prefer XSAX when small amplitude fluctuations are relevant
- **Result:** 3 TP, 0 FP, 0 FN — perfect; same indices as SAX (25, 50, 75)

📖 Ogasawara et al. (2025). *Event Detection in Time Series*. Springer. doi:10.1007/978-3-031-75941-3

```
# Constructor  
# arg1: extended alphabet (37)  
# arg2: word length (3)  
# arg3: min occurrences (3)  
model <- hmo_xsax(37, 3, 3)
```

08 – Discord Discovery with STOMP (hdis_mp)

Matrix Profile – Discord Discovery

The Matrix Profile stores, for each subsequence, the distance to its nearest neighbor. Discords = positions with the highest values (most anomalous).

1. **Compute Distances (MASS)** — for each subsequence of length w ; $O(n^2 \log n)$
2. **Build Matrix Profile** — store min distance to nearest neighbor (excluding trivial matches)
3. **Identify Discords** — positions with highest MP values = most unusual subsequences
 - Uses `hdis_mp()` instead of `hmo_mp()`
 - Dataset: ECG `mitdb102` (~7,200 points)
 - Result: 0 TP, 4 FP, 3 FN — discords don't align with clinical labels (globally anomalous $\neq$ clinically labeled)

```
# Constructor
# mode: algorithm ("stomp")
# w: window size (25)
# qtd: number of discords (10)
model <- hdis_mp(mode = "stomp",
                 w = 25, qtd = 10)
```

📄 **Citation:** Yeh et al. (2016). Matrix Profile I/II. *IEEE ICDM*.

09 – Discord Discovery with SAX (hdis_sax)

SAX – Discord Discovery

Inverts the motif logic — finds the rarest symbolic words instead of the most frequent ones, flagging unusual regions.

- Motif with SAX: most frequent words, high precision, few detections
- Discord with SAX via hdis_sax: rarest words, high sensitivity, many detections
- hdis_sax result: 121 discords, 0 TP, 121 FP, 3 FN
- hdis_mp result: 4 discords, 0 TP, 4 FP, 3 FN

📄 **Citation:** Lin et al. (2007). *Data Mining and Knowledge Discovery*.

```
# Constructor  
# arg1: alphabet size (26)  
# arg2: word/window length (25)  
model <- hdis_sax(26, 25)
```

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

